Heurísticas de Nielsen

Avaliação de Usabilidade

As 10 Heurísticas de Usabilidade

Exemplos Reais de Boas e Más Práticas

Aplicação React com Violações

Material Didático

Produção: 2026

Contents

Introdução **3**

Visibilidade do Status do Sistema **4**

Correspondência entre o Sistema e o Mundo Real **5**

Controle e Liberdade do Usuário **6**

Consistência e Padrões **7**

Prevenção de Erros **8**

Reconhecimento em vez de Memorização **9**

Flexibilidade e Eficiência de Uso **10**

Estética e Design Minimalista **11**

Ajudar os Usuários a Reconhecer, Diagnosticar e Recuperar-se de Erros **12**

Ajuda e Documentação **13**

Resumo das Heurísticas **14**

Uma Aplicação React que Viola as Heurísticas **15**

- Criando o projeto com Vite 15
- Ajustes iniciais 15
- Criando o componente GerenciadorIncorreto 16
- Integrando no App.jsx 20
- Identificação das Violações 20
- Exercício proposto 21

Referências **23**

Introdução

Jakob Nielsen é um dos profissionais mais influentes na área de usabilidade e experiência do usuário. Em 1994, juntamente com Rolf Molich, ele publicou um conjunto de dez heurísticas de usabilidade que se tornaram referência mundial para avaliação de interfaces. Essas heurísticas são princípios gerais — não regras específicas — que servem como guias para o design de interfaces intuitivas e eficientes.

A **avaliação heurística** é um método de inspeção de usabilidade no qual avaliadores examinam uma interface e julgam sua conformidade com princípios de usabilidade reconhecidos. É um método rápido e econômico, ideal para identificar problemas antes de testes com usuários.

As dez heurísticas de Nielsen são amplamente utilizadas na indústria e na academia. Nesta apostila, estudaremos cada uma delas com definições, exemplos reais de boas e más práticas (citando sites conhecidos), e ao final desenvolveremos uma pequena aplicação React que viola deliberadamente todas as heurísticas, como exercício de identificação de problemas.

Nota

As heurísticas de Nielsen não são mutuamente exclusivas. Um mesmo elemento de interface pode violar (ou respeitar) mais de uma heurística simultaneamente. Ao analisar interfaces, é natural encontrar sobreposições entre os princípios.

Visibilidade do Status do Sistema

Definição

O sistema deve sempre manter os usuários informados sobre o que está acontecendo, por meio de feedback apropriado e em tempo razoável. O usuário nunca deve ficar na dúvida sobre se sua ação foi registrada ou qual é o estado atual do sistema.

Essa heurística está diretamente ligada à confiança do usuário. Quando o sistema não comunica seu estado, o usuário fica inseguro e pode repetir ações desnecessariamente ou abandonar a tarefa.

Um bom exemplo: https://www.gmail.com/

Quando você envia um e-mail no Gmail, uma barra de notificação aparece na parte inferior da tela informando que a mensagem foi enviada com sucesso, e oferece a opção de "Desfazer". Durante o envio de anexos grandes, uma barra de progresso mostra o andamento do upload. Em ambos os casos, o usuário sabe exatamente o que está acontecendo.

Um mau exemplo: Formulários web sem indicador de carregamento

Considere um site de e-commerce em que, ao clicar no botão "Finalizar Compra", nada acontece visualmente. Não há spinner, não há barra de progresso, não há mudança de cor no botão. O sistema está processando o pagamento, mas o usuário não sabe disso. Resultado: o usuário clica novamente, potencialmente gerando uma cobrança duplicada.

Correspondência entre o Sistema e o Mundo Real

Definição

O sistema deve falar a linguagem do usuário, usando palavras, frases e conceitos familiares, em vez de termos orientados ao sistema. Deve seguir convenções do mundo real, fazendo com que a informação apareça em uma ordem natural e lógica.

Essa heurística enfatiza que a interface deve usar a linguagem e os conceitos do domínio do usuário, não o jargão técnico do desenvolvedor. Ícones, metáforas e fluxos devem fazer sentido do ponto de vista de quem usa.

Um bom exemplo: `https://www.apple.com/apple-books/`

O aplicativo Apple Books usa a metáfora de uma estante de livros. Os livros são exibidos com suas capas, podem ser "abertos" e têm páginas que "viram". Essa metáfora é intuitiva porque corresponde à experiência real do usuário com livros físicos.

Um mau exemplo: Mensagens de erro técnicas

Um site que, ao falhar em uma operação, exibe a mensagem: "`Error 500: Internal Server Error -- NullPointerException at line 342`". Essa mensagem faz sentido apenas para desenvolvedores. Para o usuário comum, não comunica nada útil sobre o que aconteceu ou o que fazer.

Controle e Liberdade do Usuário

Definição

Os usuários frequentemente escolhem funções do sistema por engano e precisam de uma "saída de emergência" claramente marcada para sair do estado indesejado sem ter que passar por um diálogo extenso. O sistema deve suportar desfazer e refazer.

Essa heurística reconhece que os seres humanos cometem erros e que o sistema deve ser tolerante a isso, oferecendo caminhos de volta fáceis e intuitivos.

Um bom exemplo: https://docs.google.com/

O Google Docs oferece Ctrl+Z para desfazer e Ctrl+Y para refazer. Além disso, mantém um histórico completo de versões do documento, acessível pelo menu. Se o usuário apagar acidentalmente uma seção inteira, pode facilmente recuperá-la.

Um mau exemplo: Assistentes (wizards) sem botão "Voltar"

Alguns formulários de cadastro divididos em múltiplas etapas não possuem um botão "Voltar". Se o usuário percebe que cometeu um erro na etapa 2 e já está na etapa 4, precisa recomeçar todo o processo do zero. Isso gera frustração e desperdício de tempo.

Consistência e Padrões

Definição

Os usuários não devem ter que se perguntar se diferentes palavras, situações ou ações significam a mesma coisa. Siga as convenções da plataforma. Elementos semelhantes devem ter aparência e comportamento semelhantes.

Consistência reduz a carga cognitiva do usuário. Quando padrões são respeitados, o conhecimento adquirido em uma parte do sistema se transfere para outras partes.

Um bom exemplo: https://www.youtube.com/

No YouTube, o ícone de polegar para cima sempre significa "gostei", em qualquer página ou contexto. Os botões de reprodução, pausa e avanço seguem o padrão universal de players de mídia. A barra de busca está sempre no topo. Essa consistência permite que novos usuários naveguem sem dificuldades.

Um mau exemplo: Sites com botões "Salvar" e "Gravar" usados alternadamente

Considere um sistema interno onde, em uma tela, o botão diz "Salvar", em outra diz "Gravar", e em outra diz "Confirmar". Todos fazem a mesma coisa. Essa inconsistência obriga o usuário a parar e pensar se as ações são realmente equivalentes ou se há diferenças sutis entre elas.

Prevenção de Erros

Definição

Melhor do que boas mensagens de erro é um design cuidadoso que previne a ocorrência do problema em primeiro lugar. O sistema deve eliminar condições propensas a erros ou verificá-las e apresentar aos usuários uma opção de confirmação antes de executar a ação.

Existem dois tipos de erros: deslizes (*slips*), que ocorrem quando o usuário pretende fazer uma ação mas acidentalmente faz outra, e enganos (*mistakes*), que ocorrem quando o modelo mental do usuário é inadequado para a tarefa.

Um bom exemplo: https://www.google.com/

O Google Search previne erros de digitação ao sugerir correções automáticas ("Você quis dizer...") e ao oferecer sugestões de autocompletar enquanto o usuário digita. Isso reduz drasticamente a chance de o usuário buscar por um termo incorreto.

Um mau exemplo: Botões "Excluir" sem confirmação

Um sistema de gerenciamento de arquivos que exclui permanentemente um arquivo ao primeiro clique no botão "Excluir", sem pedir confirmação e sem lixeira. Um único clique acidental pode resultar em perda irreversível de dados.

Reconhecimento em vez de Memorização

Definição

Minimize a carga de memória do usuário tornando objetos, ações e opções visíveis. O usuário não deve ter que se lembrar de informações de uma parte do diálogo para outra. As instruções de uso do sistema devem ser visíveis ou facilmente acessíveis quando necessário.

O cérebro humano é muito melhor em reconhecer coisas do que em lembrar delas do zero. Interfaces que exigem memorização de códigos, caminhos ou procedimentos sobrecarregam o usuário desnecessariamente.

Um bom exemplo: https://www.amazon.com.br/

A Amazon exibe o histórico de produtos visualizados recentemente, recomendações baseadas em compras anteriores e permite que o usuário veja seus pedidos passados facilmente. O usuário não precisa lembrar o que pesquisou — o sistema mostra.

Um mau exemplo: Campos de formulário sem rótulos

Um formulário de cadastro que exibe apenas campos de texto vazios, sem rótulos (*labels*), sem *placeholders* e sem indicação do formato esperado. O usuário precisa adivinhar o que digitar em cada campo, especialmente se existem campos como "Código", "Referência" ou "ID" sem qualquer explicação.

Flexibilidade e Eficiência de Uso

Definição

Aceleradores — invisíveis para o usuário novato — podem agilizar a interação para o usuário experiente, de modo que o sistema atenda tanto a usuários inexperientes quanto a experientes. Permita que os usuários personalizem ações frequentes.

Essa heurística reconhece que diferentes usuários têm diferentes níveis de experiência. Um bom sistema oferece um caminho simples para novatos e atalhos para especialistas.

Um bom exemplo: `https://code.visualstudio.com/`

O VS Code oferece uma interface gráfica completa com menus e painéis para iniciantes, mas também suporta centenas de atalhos de teclado e uma paleta de comandos (Ctrl+Shift+P) para usuários avançados. Além disso, permite personalização completa via arquivo de configuração JSON.

Um mau exemplo: Sistemas sem atalhos de teclado

Um sistema web de gestão de estoque que exige que o operador use exclusivamente o mouse para todas as operações, incluindo a inserção de dados. Mesmo para ações repetitivas realizadas centenas de vezes ao dia, não há atalhos de teclado, tornando o trabalho lento e cansativo para usuários experientes.

Estética e Design Minimalista

Definição

Os diálogos não devem conter informações irrelevantes ou raramente necessárias. Cada unidade extra de informação em um diálogo compete com as unidades relevantes de informação e diminui sua visibilidade relativa.

Minimalismo não significa falta de conteúdo, mas sim a presença apenas do conteúdo necessário. Interfaces limpas permitem que o usuário se concentre no que realmente importa.

Um bom exemplo: https://www.google.com/

A página inicial do Google é um exemplo clássico de design minimalista. Apenas o logotipo, a barra de busca e dois botões. Nenhuma informação desnecessária compete pela atenção do usuário. O foco está inteiramente na tarefa principal: buscar.

Um mau exemplo: Portais de notícias sobrecarregados

Alguns portais de notícias exibem dezenas de banners publicitários, pop-ups, barras laterais repletas de links, contadores de redes sociais, vídeos que iniciam automaticamente e notificações. O conteúdo principal — a notícia — fica perdido em meio à poluição visual.

Ajudar os Usuários a Reconhecer, Diagnosticar e Recuperar-se de Erros

Definição

As mensagens de erro devem ser expressas em linguagem simples (sem códigos técnicos), indicar precisamente o problema e sugerir uma solução de forma construtiva.

Quando erros acontecem, o sistema deve ajudar o usuário, não culpá-lo. Mensagens de erro devem ser claras, específicas e oferecer um caminho de recuperação.

Um bom exemplo: `https://github.com/`

Quando o GitHub encontra um erro 404 (página não encontrada), ele exibe uma página amigável com uma ilustração, uma mensagem clara ("This is not the web page you are looking for") e links úteis para a página inicial e para a busca. O usuário sabe que a página não existe e tem opções para continuar navegando.

Um mau exemplo: "Ocorreu um erro. Tente novamente."

Muitos sistemas exibem mensagens genéricas como "Ocorreu um erro" ou "Operação falhou" sem indicar qual foi o erro, o que o causou ou o que o usuário pode fazer para resolvê-lo. Essa mensagem é inútil e frustrante.

Ajuda e Documentação

Definição

Embora seja melhor que o sistema possa ser usado sem documentação, pode ser necessário fornecer ajuda e documentação. Qualquer informação desse tipo deve ser fácil de buscar, focada na tarefa do usuário, listar passos concretos a serem seguidos e não ser muito extensa.

Mesmo sistemas bem projetados podem necessitar de documentação. O importante é que ela seja acessível, contextual e orientada a tarefas.

Um bom exemplo: https://www.notion.so/

O Notion oferece um sistema de ajuda contextual com tooltips ao passar o mouse sobre funcionalidades, um centro de ajuda com busca integrada, guias passo a passo e vídeos curtos. Além disso, ao iniciar pela primeira vez, oferece modelos (templates) pré-prontos que servem como documentação prática.

Um mau exemplo: Sistemas sem nenhuma ajuda

Sistemas internos corporativos que não oferecem nenhum tipo de ajuda: sem tooltips, sem manual, sem FAQ, sem chat de suporte. Quando o usuário encontra uma dificuldade, a única opção é perguntar a um colega ou tentar por tentativa e erro.

Resumo das Heurísticas

Heurística	Pergunta-chave
Visibilidade do status do sistema	O usuário sabe o que está acontecendo?
Correspondência com o mundo real	A linguagem é familiar ao usuário?
Controle e liberdade do usuário	O usuário consegue desfazer e voltar?
Consistência e padrões	Elementos iguais se comportam igualmente?
Prevenção de erros	O sistema evita que o erro aconteça?
Reconhecimento em vez de memorização	As opções estão visíveis?
Flexibilidade e eficiência de uso	Há atalhos para usuários experientes?
Estética e design minimalista	Há apenas o necessário na tela?
Reconhecer, diagnosticar e recuperar erros	As mensagens de erro ajudam?
Ajuda e documentação	Existe ajuda acessível e objetiva?

Uma Aplicação React que Viola as Heurísticas

Vamos desenvolver uma pequena aplicação React que viola deliberadamente as 10 heurísticas de Nielsen. Trata-se de um gerenciador de tarefas com diversos problemas de usabilidade. O objetivo é que, ao identificar cada violação no código, o aluno compreenda na prática o impacto de ignorar cada heurística.

Criando o projeto com Vite

O primeiro passo é criar a aplicação com o comando a seguir.

```
npm create vite@latest aula-ihc-heuristicas-nielsen -- --template react
```

É possível que apareça a sugestão de uso da versão experimental do Vite 8. Como estamos em um ambiente de aprendizado, façamos uso dela. Também é possível que apareça a sugestão de já instalar as dependências com o npm e colocar o projeto em execução. Diga que deseja fazê-lo. Caso não apareça a opção de instalar as dependências e executar o projeto, você pode fazê-lo com:

```
cd aula-ihc-heuristicas-nielsen
npm i
npm run dev
```

A seguir, basta abrir o navegador e visitar `localhost:5173`.

Ajustes iniciais

Faça os seguintes ajustes no projeto:

- Apague o arquivo `App.css`
- Apague o arquivo `index.css`
- Abra o arquivo `main.jsx` e apague a linha que importa o `index.css`
- Substitua o conteúdo do arquivo `App.jsx` pelo código a seguir

```
export default () => {
  return (
    <div>
      <h1>Olá, mundo!</h1>
    </div>
  );
}
```

Verifique no navegador que a aplicação exibe "Olá, mundo!".

Criando o componente GerenciadorIncorreto

Crie um arquivo chamado `GerenciadorIncorreto.jsx` na pasta `src`. Vamos construí-lo em etapas. Comece com a estrutura básica e o estado do componente.

```
import { useState } from 'react';

export const GerenciadorIncorreto = () => {
  const [registro, setRegistro] = useState('');
  const [tarefas, setTarefas] = useState([]);
  const [erro, setErro] = useState('');
  const [contadorCliques, setContadorCliques] = useState(0);

  // As funções serão adicionadas a seguir

  return null; // Temporário
};
```

Agora, vamos adicionar a função de inserir uma tarefa. Observe as violações indicadas nos comentários.

```
import { useState } from 'react';

export const GerenciadorIncorreto = () => {
  const [registro, setRegistro] = useState('');
  const [tarefas, setTarefas] = useState([]);
  const [erro, setErro] = useState('');
  const [contadorCliques, setContadorCliques] = useState(0);

```

```
  // VIOLAÇÃO H5 - Prevenção de erros:
  // Permite inserir tarefas vazias. Quando o texto tem
  // entre 1 e 2 caracteres, exibe erro técnico.
  const handleInserir = () => {
    setContadorCliques(c => c + 1);
    if (registro.length > 0 && registro.length < 3) {
      // VIOLAÇÃO H9 - Reconhecer/diagnosticar erros:
      // Mensagem técnica e incompreensível para o usuário
      setErro('ERR_0x003F: VALIDATION_CONSTRAINT_FAIL');
      return;
    }
    setErro('');
    const nova = {
      id: Date.now(),
      texto: registro || '(null)',
      hash: Math.random().toString(36).slice(2, 8),
      ts: Date.now()
    };
    setTarefas([...tarefas, nova]);
    // VIOLAÇÃO H1 - Visibilidade do status:
    // Nenhum feedback visual de que a tarefa foi adicionada.
    // O campo é limpo silenciosamente.
    setRegistro('');
  };

  return null; // Temporário
};
```

Agora, adicionamos a função de remover e o JSX do componente. Observe todas as violações comentadas.

```
import { useState } from 'react';

export const GerenciadorIncorreto = () => {
  const [registro, setRegistro] = useState('');
  const [tarefas, setTarefas] = useState([]);
  const [erro, setErro] = useState('');
  const [contadorCliques, setContadorCliques] = useState(0);

  const handleInserir = () => {
    setContadorCliques(c => c + 1);
    if (registro.length > 0 && registro.length < 3) {
      setErro('ERR_0x003F: VALIDATION_CONSTRAINT_FAIL');
      return;
```

```
14        }
15        setErro('');
16        const nova = {
17          id: Date.now(),
18          texto: registro || '(null)',
19          hash: Math.random().toString(36).slice(2, 8),
20          ts: Date.now()
21        };
22        setTarefas([...tarefas, nova]);
23        setRegistro('');
24      };
25
26      // VIOLAÇÃO H3 - Controle e liberdade do usuário:
27      // Remove imediatamente sem confirmação
28      // e sem possibilidade de desfazer (Ctrl+Z)
29      const handleRemover = (id) => {
30        setTarefas(tarefas.filter(t => t.id !== id));
31      };
32
33      return (
34        <div style={{ padding: '30px', backgroundColor: '#f5f5f5' }}>
35
36          {/* VIOLAÇÃO H2 - Correspondência com o mundo real: */}
37          {/* Usa jargão técnico (SQL) em vez de linguagem */}
38          {/* familiar ao usuário */}
39          <h2>INSERT INTO task_db</h2>
40
41          {/* VIOLAÇÃO H6 - Reconhecimento em vez de memorização: */}
42          {/* Campo sem label e sem placeholder. O usuário */}
43          {/* precisa adivinhar o que deve digitar. */}
44          <div style={{ marginBottom: '10px' }}>
45            <input
46              type="text"
47              value={registro}
48              onChange={(e) => setRegistro(e.target.value)}
49              style={{
50                border: 'none', outline: 'none',
51                padding: '5px', width: '220px'
52              }}
53            />
54          </div>
55
56          {erro && (
57            <p style={{ color: 'gray', fontSize: '10px' }}>{erro}</p>
58          )}
59
60          <div style={{
```

```
        display: 'flex', gap: '10px', marginBottom: '20px'
      }}>
        {/* VIOLAÇÃO H4 - Consistência e padrões: */}
        {/* Os dois "botões" têm estilos completamente */}
        {/* diferentes: um é botão, outro é link */}
        <button onClick={handleInserir} style={{
          backgroundColor: 'green', color: 'white',
          padding: '10px 30px', borderRadius: '20px',
          border: 'none', fontSize: '16px', cursor: 'pointer'
        }}>
          COMMIT
        </button>

        <span onClick={() => { setRegistro(''); setErro(''); }}
          style={{
            color: 'blue', textDecoration: 'underline',
            cursor: 'pointer', fontSize: '11px',
            alignSelf: 'flex-end'
          }}>
          ROLLBACK
        </span>
      </div>

      {/* VIOLAÇÃO H8 - Estética e design minimalista: */}
      {/* Informação irrelevante (contador de cliques) */}
      {/* exibida de forma proeminente */}
      <p>Eventos de clique registrados: {contadorCliques}</p>

      {/* VIOLAÇÃO H7 - Flexibilidade e eficiência: */}
      {/* Sem atalhos de teclado, sem busca, sem filtro */}

      {/* VIOLAÇÃO H10 - Ajuda e documentação: */}
      {/* Nenhum tooltip, nenhuma ajuda, nenhuma */}
      {/* instrução disponível em lugar algum */}

      <ul style={{ listStyle: 'none', padding: 0 }}>
        {tarefas.map(t => (
          <li key={t.id} style={{
            padding: '8px', marginBottom: '4px',
            backgroundColor: '#fff', display: 'flex',
            justifyContent: 'space-between'
          }}>
            <div>
              <strong>{t.texto}</strong>
              {/* VIOLAÇÃO H8 - Estética e design minimalista: */}
              {/* Metadados técnicos exibidos ao usuário */}
              <br/>
```

```
              <span style={{ fontSize: '9px', color: '#aaa' }}>
                ID:{t.id} | Hash:{t.hash} | TS:{t.ts}
              </span>
            </div>
            <button onClick={() => handleRemover(t.id)} style={{
              backgroundColor: 'red', color: 'white',
              border: 'none', padding: '4px 10px'
            }}>
              DROP
            </button>
          </li>
        ))}
      </ul>
    </div>
  );
};
```

Integrando no App.jsx

Agora, atualize o arquivo `App.jsx` para utilizar o componente que acabamos de criar.

```
import { GerenciadorIncorreto }
  from "./GerenciadorIncorreto";

export default () => {
  return (
    <div>
      <GerenciadorIncorreto />
    </div>
  );
}
```

Abra o navegador e observe o resultado. Você verá um gerenciador de tarefas com diversos problemas de usabilidade.

Identificação das Violações

A tabela a seguir resume todas as violações presentes na aplicação.

Heurística violada	Como se manifesta na aplicação
H1 — Visibilidade do status	Nenhum feedback ao adicionar tarefa. O campo é limpo silenciosamente.
H2 — Mundo real	Jargão SQL: "INSERT INTO", "COMMIT", "ROLLBACK", "DROP".
H3 — Controle e liberdade	Remoção imediata sem confirmação e sem desfazer.
H4 — Consistência	Botão "COMMIT" e link "ROLLBACK" têm estilos e formatos completamente diferentes para ações do mesmo nível.
H5 — Prevenção de erros	Permite inserir tarefas vazias sem validação.
H6 — Reconhecimento	Campo de entrada sem rótulo e sem placeholder.
H7 — Flexibilidade	Sem atalhos de teclado, sem busca, sem filtros.
H8 — Estética minimalista	Metadados técnicos (ID, Hash, TS) e contador de cliques exibidos ao usuário.
H9 — Recuperação de erros	Mensagem "ERR_0x003F: VALIDATION_CONSTRAINT_FAIL" incompreensível.
H10 — Ajuda	Nenhum tooltip, instrução ou documentação disponível.

Exercício proposto

Como exercício, crie uma nova versão do componente chamada `GerenciadorCorreto.jsx` que corrija todas as violações identificadas. Algumas sugestões:

- Adicione uma mensagem de sucesso ao inserir uma tarefa (H1).
- Substitua o jargão técnico por linguagem natural como "Adicionar tarefa" (H2).
- Adicione um diálogo de confirmação antes de remover e permita desfazer (H3).
- Padronize os estilos dos botões (H4).
- Valide o campo antes de permitir a inserção (H5).
- Adicione labels e placeholders nos campos (H6).

- Adicione suporte a Enter para submeter e atalhos de teclado (H7).
- Remova os metadados técnicos e o contador de cliques (H8).
- Exiba mensagens de erro claras, como “O nome da tarefa deve ter pelo menos 3 caracteres” (H9).
- Adicione tooltips nos botões e um texto de ajuda inicial (H10).

Referências

NIELSEN, Jakob. *10 Usability Heuristics for User Interface Design*. Nielsen Norman Group, 1994 (atualizado em 2024). Disponível em: https://www.nngroup.com/articles/ten-usability-heuristics/. Acesso em: fev. 2026.

NIELSEN, Jakob; MOLICH, Rolf. Heuristic evaluation of user interfaces. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 1990. p. 249–256.

NIELSEN, Jakob. *Usability Engineering*. Boston: Academic Press, 1993.

REACT. React. [S. l.]: Meta, [2026]. Disponível em: https://react.dev/. Acesso em: fev. 2026.